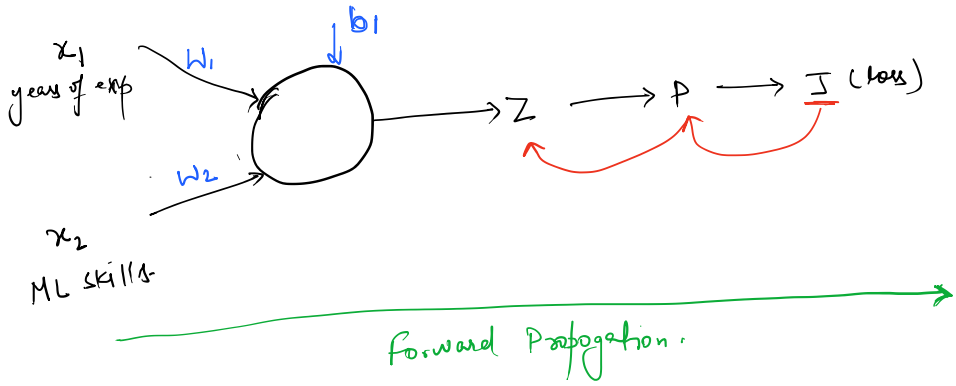
N-Layer Neural Networks

Tuesday, 22 September 2026 5:31 PM



$Z = W_1 x_1 + W_2 x_2 + b_1$

$P = \text{sigmoid}(Z)$

$Y = \text{Ground truth.}$

$J = P - Y$

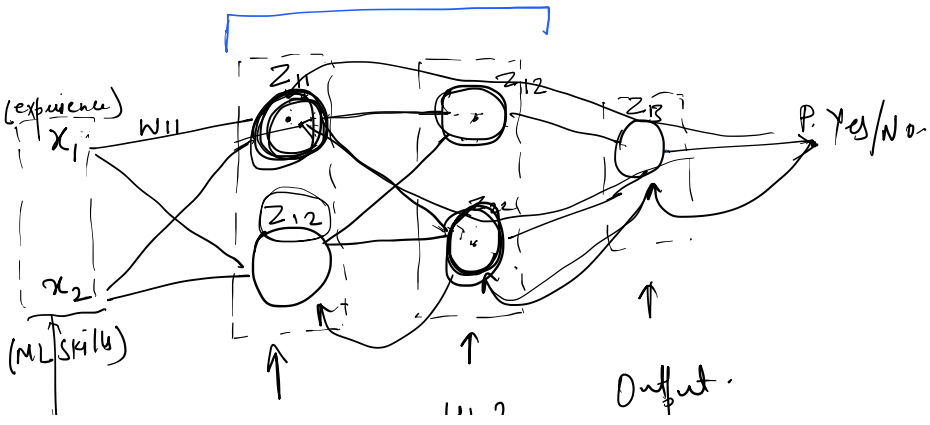
	A	B	C
Y	0	1	0
P	0.2	0.7	0.1

$P - Y = [0.2 \quad -0.3 \quad 0.1]$. → dZ $(P - Y)$.

> "Say I give you a single layer of neurons — and you can have as many neurons in it as you want. A hundred. A million. Is there any pattern in the data that a layer like that could *never* learn, no matter how wide it gets?"

Chat poll:

- A) No — enough neurons in one layer can learn anything
- B) Yes — some patterns are impossible for a single layer, no matter the width
- C) Depends entirely on the learning rate

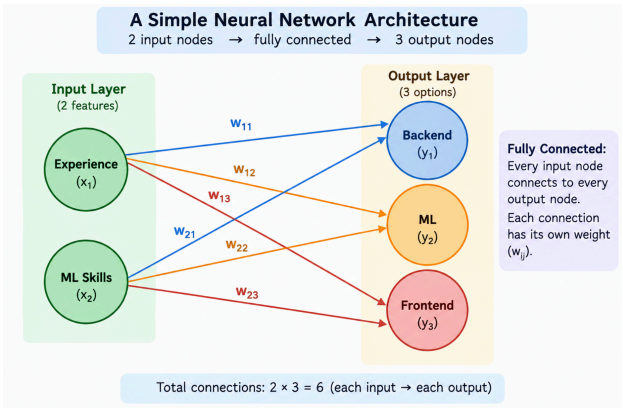


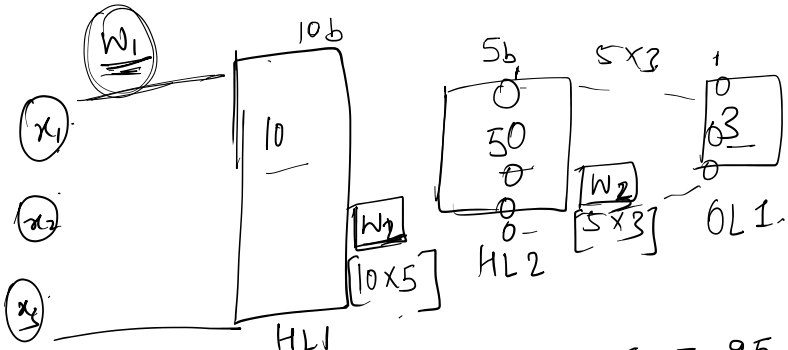
Input HL1 HL2 ...

Hidden layers are responsible for feature extraction.

Hidden layer 1 → Extracts high level or simple features. Detect vertical and horizontal edges.

Hidden layer 4 → They start detecting shapes.

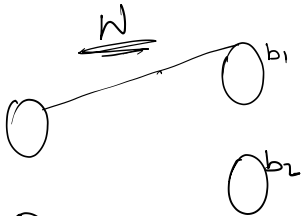


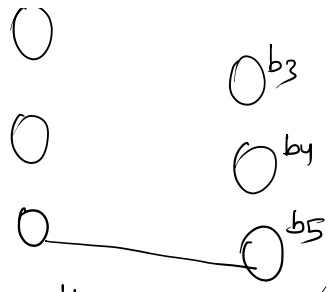


Weights = 30 + 50 + 15 = 95

bias = 18.

Total parameters = 113.





4 X 5 = 20 weights.

5 bias

20 + 5 = 25 parameters.

Quiz2 :

T1 T2 T3

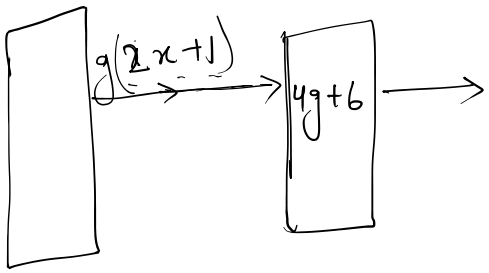
$Y = [0 \quad 1 \quad 0]$

$P = [0.5 \quad 0.2 \quad 0.3]$

$P - Y = [0.5-0 \;,\; 0.2-1 \;,\; 0.3-0]$

$= [0.5 \;,\; -0.8 \;,\; 0.3]$.

Softmax → why do we use e^2



$g = 2x +$ → linear function.

$4g + 6$

$4(2x + 1) + 6$

$8x + 4 + 6$

$8x + 10$;

* Sigmoid → 0 - 1 (z)

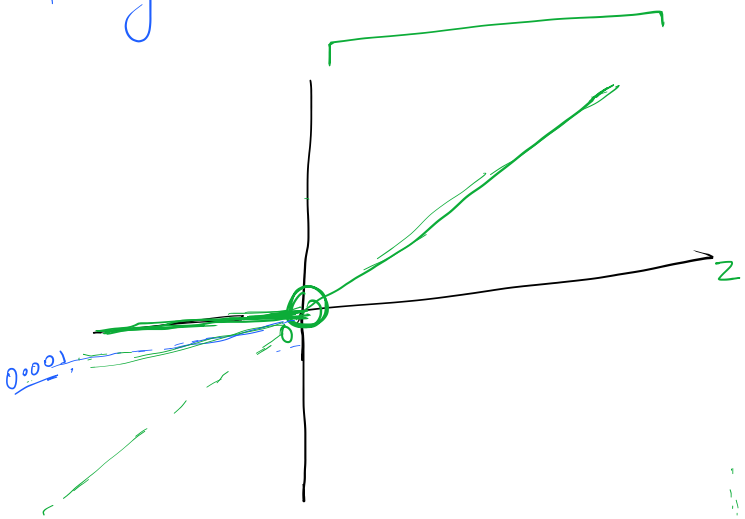
0.1

$0.1 * 0.1 * 0.1 \ldots 0.1$ ($d_1 * d_2 * d_3 \ldots d_{50}$)

$(0.1)^{50}$ (Vanishing Gradients)

* ReLU → Rectified Linear Unit

Mostly used in hidden layers.



0.00000001

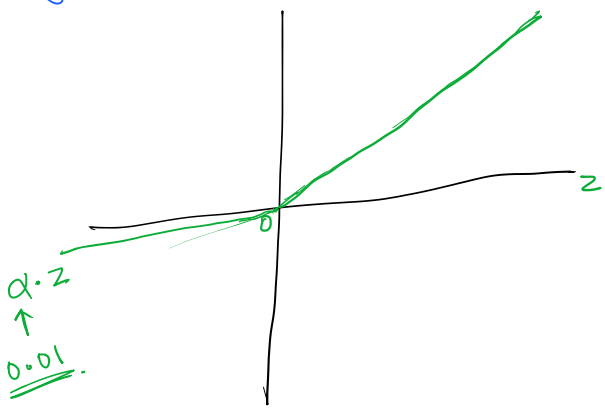
$ReLU = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z <= 0 \end{cases}$

$ReLU = \max(z, 0)$

Tanh

Problem with ReLU is dead neuron.

* Leaky ReLU



$1000 \times 0.01 = 10$

$Leaky\ ReLU = \begin{cases} z & \text{if } z \geq 0 \\ \ldots & \ldots < 0 \end{cases}$

$\alpha z \quad y \quad 2 \; 2 \; 0$

0.01

Exclusive OR -

The XOR Pattern (Hiring Decision)

Hire if exactly one of the inputs is 1, otherwise do not hire.

Input → Output

Experience (x_1)	ML Skills (x_2)	Decision (y)
0	0	No
0	1	Hire
1	0	Hire
1	1	No

underqualified.

overqualified

2 × 2 Grid (XOR Pattern)

= Hire
= No

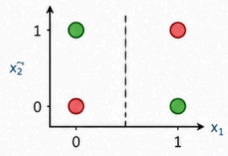
The "Hire" points (0, 1) and (1, 0) are on opposite corners. No single straight line can separate them from the "No" points (0, 0) and (1, 1).

OR -

x_1	x_2	OR	XOR
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	0

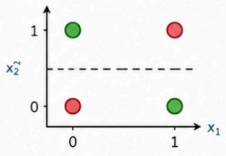
Trying different straight lines (all fail)

Vertical line ($x_1 = 0.5$)



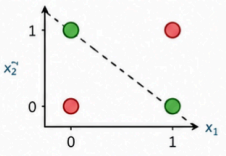
Does not separate: both sides contain a mix of Hire and No.

Horizontal line ($x_2 = 0.5$)



Does not separate: both sides contain a mix of Hire and No.

Diagonal line ($x_1 + x_2 = 1$)



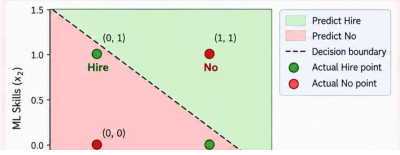
Does not separate: "Hire" points end up on different sides.

Conclusion: No single straight line can perfectly separate the **"Hire"** points from the **"No"** points. **The XOR problem is not linearly separable.**

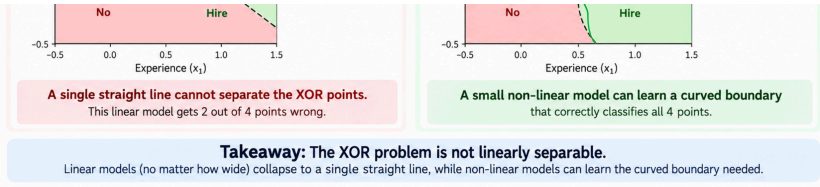
Decision Boundaries on the XOR Problem

A linear model can't solve XOR, but a small non-linear model can.

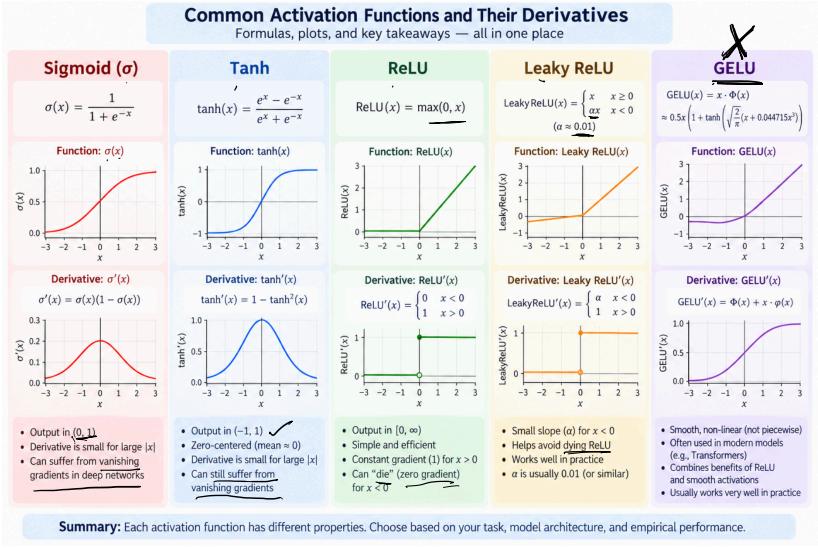
Wide Linear Model (single straight boundary)



Tiny Non-linear Model (curved boundary)



* Both Sigmoid & Tanh suffer with vanishing gradient problem.



inclusive → $[0, \infty)$ ← exclusive